



UNIVERSITÀ DI PADOVA

Dipartimento di Matematica

Laurea Triennale in Informatica

Anno Accademico 2025/2026

Progetto di Basi di Dati

Base di Dati di un Sistema Informativo
per la Gestione Alberghiera

1 Abstract

Il progetto riguarda la progettazione di una base di dati per la gestione delle informazioni principali di un hotel. Il sistema modella le strutture alberghiere, le camere, i clienti, il personale, le prenotazioni, i servizi e i pagamenti. Il contesto considerato è quello di una struttura alberghiera che deve organizzare in modo ordinato le proprie attività quotidiane.

La base di dati permette di registrare i dati degli hotel, come nome, indirizzo, città, regione, stelle e telefono. Consente inoltre di gestire le camere disponibili nella struttura, specificando numero, tipologia, prezzo e stato corrente. Una funzione fondamentale è la gestione delle prenotazioni, attraverso il collegamento tra cliente, camera in base periodo di soggiorno. Il sistema permette quindi di controllare le date di check-in e check-out, il numero di ospiti e lo stato della prenotazione. Vengono inoltre modellati i clienti e il personale dell'hotel, distinguendo le informazioni comuni da quelle specifiche dei ruoli. Tra il personale è previsto anche il direttore della struttura, associato all'hotel che gestisce. Un'altra funzione importante riguarda i servizi offerti, come Wi-Fi, colazione, parcheggio, piscina o servizio in camera. La base di dati permette di associare ogni servizio agli hotel in cui è disponibile. Infine, il sistema gestisce i pagamenti collegati alle prenotazioni, registrando importo, data, metodo e stato del pagamento. L'obiettivo è rendere più semplice, chiara ed efficiente la gestione dell'hotel.

2 Analisi dei Requisiti

Questa sezione descrive i requisiti che la base di dati deve soddisfare per supportare la gestione di hotel.

Hotel

Ogni hotel è identificato da un codice univoco e contiene le seguenti informazioni.

- Codice univoco che identifica in modo univoco ogni struttura.
- Nome dell'hotel.
- Indirizzo.
- Città.
- Regione.
- Numero di stelle.
- Numero di telefono.
- Data di apertura.

Gli hotel si suddividono in hotel indipendenti e hotel appartenenti a una catena.

Hotel Indipendenti

Per gli hotel indipendenti, oltre alle informazioni generali, vengono registrati i seguenti dati.

- Nome del proprietario.
- Categoria della struttura, ad esempio boutique hotel, resort o albergo.

Hotel di Catena

Per gli hotel appartenenti a una catena, oltre alle informazioni di base, si registrano ulteriori informazioni.

- Nome della catena alberghiera.
- Sede centrale della catena.

Camere

Le camere costituiscono le unità principali di ogni hotel e sono caratterizzate dai seguenti attributi.

- Numero della camera, univoco all'interno dell'hotel.
- Tipologia, ad esempio singola, doppia, suite o familiare.
- Prezzo per notte.
- Stato attuale, ad esempio libera, occupata o in manutenzione.

Persone

Nel sistema viene introdotta l'entità Persona, che possiede le informazioni anagrafiche comuni a clienti e dipendenti.

Ogni persona è identificata da un codice fiscale univoco ed è descritta dai seguenti attributi:

- Codice fiscale
- Nome
- Cognome
- Numero di telefono
- Indirizzo Email (se noto)

L'entità Persona viene specializzata in due entità figlie: Cliente e Dipendente che condividono gli stessi dati anagrafici di base.

Clienti

I clienti effettuano soggiorni presso le strutture alberghiere.

L'entità Cliente, oltre agli attributi ereditati da Persona, include ulteriori informazioni specifiche.

- Data di registrazione, cioè la data del primo inserimento nel sistema
- Nazionalità

Dipendenti

Ogni hotel impiega personale per la gestione della struttura. Ogni dipendente è identificato dai seguenti dati.

- Codice dipendente.
- Data di assunzione.
- Stipendio.

Direttori

Il direttore è modellato come specializzazione dell'entità Dipendente che ha responsabilità di gestione e coordinamento della struttura alberghiera.

L'entità Direttore eredita gli attributi di Dipendente e aggiunge le seguenti informazioni specifiche:

- Data di nomina.
- Bonus di responsabilità.

Ogni direttore dirige un solo hotel, mentre ogni hotel deve avere esattamente un direttore.

Prenotazioni

Ogni prenotazione associa un cliente a una camera per un determinato periodo e include le seguenti informazioni.

- Codice della prenotazione.
- Data di prenotazione.
- Data di check-in.
- Data di check-out.
- Numero di ospiti.
- Stato della prenotazione, ad esempio confermata, in corso, conclusa o annullata.

Una prenotazione è riferita a una sola camera e a un solo cliente, mentre una camera può essere associata a più prenotazioni in periodi distinti.

Non è possibile che due prenotazioni attive si sovrappongano per la stessa camera nello stesso intervallo di tempo.

Servizi

Ogni hotel può offrire diversi servizi ai propri clienti, come Wi-Fi, colazione, parcheggio, piscina, palestra, servizio in camera.

Ogni servizio è identificato dalle seguenti informazioni.

- Codice univoco del servizio.
- Nome del servizio.
- Descrizione.
- Costo aggiuntivo, se previsto.

Uno stesso servizio può essere offerto da più hotel, e ogni hotel può offrire più servizi.

Pagamenti

Ogni prenotazione può essere associata a uno o più pagamenti, ognuno dei quali registra le seguenti informazioni.

- Codice pagamento.
- Data del pagamento.
- Importo.
- Metodo di pagamento, ad esempio contanti, carta o bonifico.
- Stato del pagamento, ad esempio effettuato, in attesa o rimborsato.

Ogni pagamento è riferito a una sola prenotazione, mentre una prenotazione può essere saldata in una o più soluzioni.

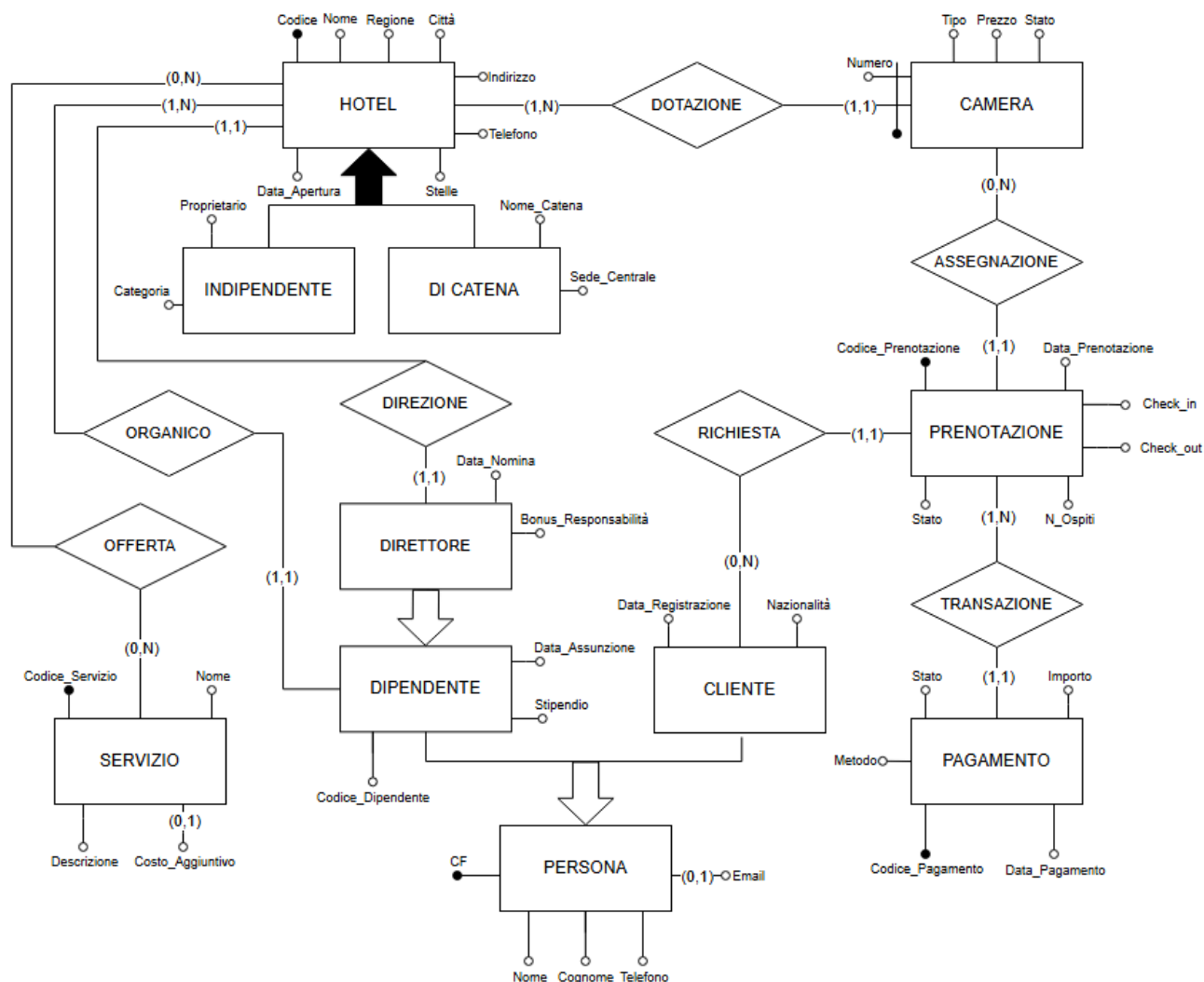


Figura 1: Diagramma E-R della base di dati relativa alla gestione di hotel.

3 Progettazione Concettuale

La Figura 1 mostra il diagramma Entità-Relazione (E-R) che riassume i requisiti descritti nella Sezione 2.

Esistono esattamente due tipi di hotel, indipendenti e di catena. Gli hotel indipendenti sono caratterizzati da un proprietario, una data di apertura e una categoria della struttura. Gli hotel di catena appartengono invece a una catena alberghiera, della quale vengono registrati il nome della catena e la sede centrale. Una stessa catena può gestire più hotel, mentre ogni hotel di catena appartiene a una sola catena.

Ogni hotel è costituito da un insieme di camere. Poiché una camera non avrebbe significato al di fuori della struttura a cui appartiene, il concetto di Camera è modellato come entità dipendente da Hotel. In particolare, il numero della camera è univoco soltanto all'interno di uno stesso hotel. Per ogni camera si vogliono registrare tipologia, prezzo per notte e stato attuale.

L'analisi dei requisiti introduce l'entità PERSONA per raccogliere le informazioni anagrafiche comuni a clienti e dipendenti. Il concetto viene quindi specializzato in CLIENTE e DIPENDENTE. I clienti rappresentano gli ospiti delle strutture alberghiere e possono effettuare più prenotazioni nel

tempo, anche presso hotel diversi. I dipendenti rappresentano invece il personale che lavora nella struttura e risultano sempre associati a un solo hotel.

Il direttore è modellato come un caso particolare di DIPENDENTE, poiché rappresenta un ruolo specifico del personale con responsabilità di gestione e coordinamento. Ogni direttore dirige un solo hotel, mentre ogni hotel deve avere esattamente un direttore.

Le prenotazioni rappresentano il legame tra cliente e camera. Ogni prenotazione si riferisce a un solo cliente e a una sola camera, mentre sia un cliente sia una camera possono comparire in più prenotazioni nel tempo. Per ogni prenotazione vengono registrate le informazioni necessarie alla gestione del soggiorno, come data di prenotazione, periodo di permanenza, numero di ospiti e stato. Si noti che il modello E-R non consente di esprimere direttamente il vincolo che impedisce la sovrapposizione di due prenotazioni attive per la stessa camera nello stesso intervallo temporale.

Ogni prenotazione può essere associata a uno o più pagamenti. Ogni pagamento si riferisce a una sola prenotazione e ne registra importo, data, metodo e stato. Questa scelta consente di rappresentare sia pagamenti effettuati in un'unica soluzione sia pagamenti suddivisi in più transazioni.

Infine, ogni hotel può offrire diversi servizi ai propri clienti. Uno stesso servizio può essere disponibile in più hotel, e ogni hotel può offrire più servizi.

La Tabella 1 riassume tutte le entità e relazioni individuate dall'analisi dei requisiti e rappresentate nel diagramma E-R, con i relativi attributi. Per ciascuna entità viene inoltre indicato l'identificatore, che nel caso di Camera dipende anche dalla relazione con Hotel.

Tabella 1: Dizionario dei dati

Entità	Descrizione	Attributi	Identificatore
Hotel	Struttura alberghiera	<i>Codice, Nome, Indirizzo, Città, Regione, Stelle, Telefono, Data_Apertura</i>	<i>Codice</i>
Indipendente	Hotel indipendente	<i>Proprietario, Categoria</i>	
Di catena	Hotel di catena alberghiera	<i>Nome_Catena, Sede_Centrale</i>	
Camera	Camera di hotel	<i>Numero, Tipo, Prezzo, Stato</i>	<i>Numero. Relazione "Part-Of"</i>
Persona	Persona registrata nel sistema	<i>CF, Nome, Cognome, Telefono, Email</i>	<i>CF</i>
Cliente	Cliente dell'hotel	<i>Data_Registrazione, Nazionalità</i>	
Dipendente	Personale dell'hotel	<i>Codice_Dipendente, Data_Assunzione, Stipendio</i>	
Direttore	Direttore dell'hotel	<i>Data_Nomina, Bonus_Responsabilità</i>	
Prenotazione	Prenotazione di soggiorno	<i>Codice_Prenotazione, Data_Prenotazione, Check_In, Check_Out, N_Ospiti, Stato</i>	<i>Codice_Prenotazione</i>
Servizio	Servizio offerto dall'hotel	<i>Codice_Servizio, Nome, Descrizione, Costo_Aggiuntivo</i>	<i>Codice_Servizio</i>
Pagamento	Pagamento di una prenotazione	<i>Codice_Pagamento, Data_Pagamento, Importo, Metodo, Stato</i>	<i>Codice_Pagamento</i>

(a) Entità

Relazione	Descrizione	Componente	Cardinalità
Dotazione	Appartenenza di una camera a un hotel	Hotel, Camera	(1,N)
Organico	Impiego di un dipendente in un hotel	Hotel, Dipendente	(1,N)
Direzione	Direzione di un hotel	Hotel, Direttore	(1,1)
Richiesta	Prenotazione effettuata da un cliente	Cliente, Prenotazione	(1,N)
Assegnazione	Assegnazione di una prenotazione a una camera	Camera, Prenotazione	(1,N)
Offerta	Servizi offerti da un hotel	Hotel, Servizio	(N,N)
Transazione	Pagamento associato a una prenotazione	Prenotazione, Pagamento	(1,N)

(b) Relazioni

4 Progettazione Logica

In questa sezione viene descritto il passaggio dal modello concettuale al modello logico, con l'obiettivo di organizzare i dati in una forma più adatta alla realizzazione della base di dati. Per prima cosa si analizzano le eventuali ridondanze presenti nel modello, così da capire quali informazioni conviene mantenere e quali invece possono essere ricavate da altri dati. Successivamente, lo schema viene semplificato eliminando le generalizzazioni e riorganizzando le entità e le relazioni in una struttura compatibile con il modello relazionale. Infine, viene presentato il diagramma ristrutturato con le principali modifiche introdotte.

4.1 Analisi delle Ridondanze

Nel modello concettuale proposto, l'attributo *Stato* di CAMERA (ad esempio libera, occupata o in manutenzione) introduce una possibile ridondanza. In particolare, una camera può essere considerata occupata se esiste una prenotazione attiva che la riguarda, cioè una prenotazione valida per il periodo attuale e non annullata. Questo significa che lo stato della camera può essere ricavato controllando le prenotazioni, senza doverlo salvare per forza come informazione separata.

Tuttavia, l'attributo viene aggiornato frequentemente dal personale della struttura, in particolare durante le operazioni di check-in, check-out e gestione delle camere (ad esempio per segnare una camera come disponibile o in manutenzione), ed è consultato molto spesso per verificare rapidamente la disponibilità corrente delle stanze. Questo si riassume nelle seguenti due operazioni principali:

- **Operazione 1 (circa 80 al giorno):** aggiornamento dello stato di una camera in seguito a check-in, check-out o manutenzione.
- **Operazione 2 (circa 500 al giorno):** visualizzazione rapida dello stato corrente di una camera per controllarne la disponibilità.

Assumendo i seguenti volumi nella base di dati:

Concetto	Costrutto	Volume
HOTEL	E	20
CAMERA	E	600
PRENOTAZIONE	E	30000

la seguente analisi serve per stabilire se sia conveniente o meno mantenere l'attributo ridondante *Stato* in CAMERA.

CON RIDONDANZA

Analizziamo prima il costo complessivo mantenendo l'attributo ridondante.

- **Operazione 1:**

Concetto	Costrutto	Accessi	Tipo	Frequenza
CAMERA	E	1	L	80
CAMERA	E	1	S	80

- **Operazione 2:**

Concetto	Costrutto	Accessi	Tipo	Frequenza
CAMERA	E	1	L	500

Assumendo costo doppio per gli accessi in scrittura:

$$\text{Costo Totale} = 80 \times (1 + 2) + 500 = 740$$

SENZA RIDONDANZA

Analizziamo ora il costo complessivo senza memorizzare l'attributo *Stato*, calcolandolo ogni volta dalle prenotazioni.

- **Operazione 1:**

Concetto	Costrutto	Accessi	Tipo	Frequenza
PRENOTAZIONE	E	1	S	80

- **Operazione 2**

(assumendo in media circa $30000 / 600 = 50$ prenotazioni per camera):

Concetto	Costrutto	Accessi	Tipo	Frequenza
CAMERA	E	1	L	500
PRENOTAZIONE	E	50	L	500

Assumendo costo doppio per gli accessi in scrittura:

$$\text{Costo Totale} = 80 \times 2 + 500 \times (1 + 50) = 25660$$

L'analisi mostra che calcolare ogni volta lo stato della camera a partire dalle prenotazioni comporterebbe un numero di accessi molto elevato, soprattutto perché la disponibilità delle camere viene consultata frequentemente dal personale e dai sistemi di prenotazione. Al contrario, mantenere l'attributo ridondante *Stato* in CAMERA consente di ridurre sensibilmente il costo complessivo delle operazioni.

Si sceglie quindi di mantenere l'attributo ridondante *Stato* nell'entità CAMERA, poiché il vantaggio in termini di efficienza nelle interrogazioni supera il costo aggiuntivo dovuto agli aggiornamenti.

4.2 Eliminazione delle Generalizzazioni

Le generalizzazioni introdotte nel modello concettuale vengono eliminate per semplificare il passaggio al modello relazionale e rendere lo schema più facile da implementare.

Nel modello E-R è naturale rappresentare concetti come HOTEL INDIPENDENTE / HOTEL DI CATENA oppure CLIENTE / DIPENDENTE / DIRETTORE tramite generalizzazioni, perché aiutano a descrivere bene il dominio. Tuttavia, nel modello relazionale queste strutture non esistono direttamente quindi devono essere tradotte in una o più tabelle.

Per questo motivo si sceglie di eliminare le generalizzazioni accorpendo le specializzazioni nelle entità principali. In questo modo si ottiene uno schema più semplice, con meno tabelle e meno collegamenti tramite chiavi esterne. Questa scelta rende anche più immediata l'implementazione in SQL, come si vede nello schema relazionale finale, dove le entità ristrutturare HOTEL e PERSONA vengono tradotte nelle tabelle Hotel e Persona con attributi specifici delle varie tipologie.

Nel caso di HOTEL, mantenere tabelle separate per hotel indipendenti e hotel di catena avrebbe richiesto più relazioni e avrebbe complicato i collegamenti con camere, servizi, prenotazioni e direttore. Accorpendo tutto in una sola tabella HOTEL, gli attributi comuni vengono scritti una sola volta, mentre gli attributi specifici vengono compilati solo quando servono. L'attributo *Tipo* permette di distinguere se l'hotel è indipendente o appartiene a una catena.

Nel caso di PERSONA, l'accorpamento è ancora più utile perché una stessa persona può avere più ruoli nel sistema. Ad esempio, una persona può essere cliente e, in un altro contesto, anche dipendente; inoltre un direttore è comunque un dipendente con attributi aggiuntivi. Tenere tutto in un'unica entità evita duplicazioni dei dati anagrafici, come nome, cognome, telefono ed email, e permette di rappresentare i ruoli tramite la presenza o assenza degli attributi specifici.

Lo svantaggio principale di questa scelta è la presenza di alcuni valori nulli, perché non tutti gli attributi sono applicabili a tutte le istanze. Ad esempio, un hotel indipendente non avrà *Nome_Catena* e *Sede_Centrale*, mentre un hotel di catena non avrà *Proprietario* e *Categoria*. Allo stesso modo, un cliente non avrà *Stipendio* o *Data_Assunzione*. Tuttavia, nel progetto questo compromesso è considerato accettabile perché i valori nulli introdotti sono controllati e facilmente interpretabili: essi riguardano solo attributi specifici di una determinata tipologia di hotel o di persona. La distinzione tra i casi viene infatti gestita tramite attributi discriminanti, come *Tipo* per HOTEL, e tramite vincoli di integrità che impediscono combinazioni incoerenti di valori. In questo modo, pur avendo alcuni attributi opzionali, lo schema evita la proliferazione di tabelle specializzate e riduce il numero di JOIN necessari nelle interrogazioni. La soluzione risulta quindi più semplice da implementare in SQL, più leggibile nello schema relazionale e più pratica da usare nelle query principali del progetto.

Il diagramma E-R ristrutturato, mostrato in Figura 2, rappresenta il risultato delle modifiche introdotte rispetto alla versione iniziale presentata in Figura 1.

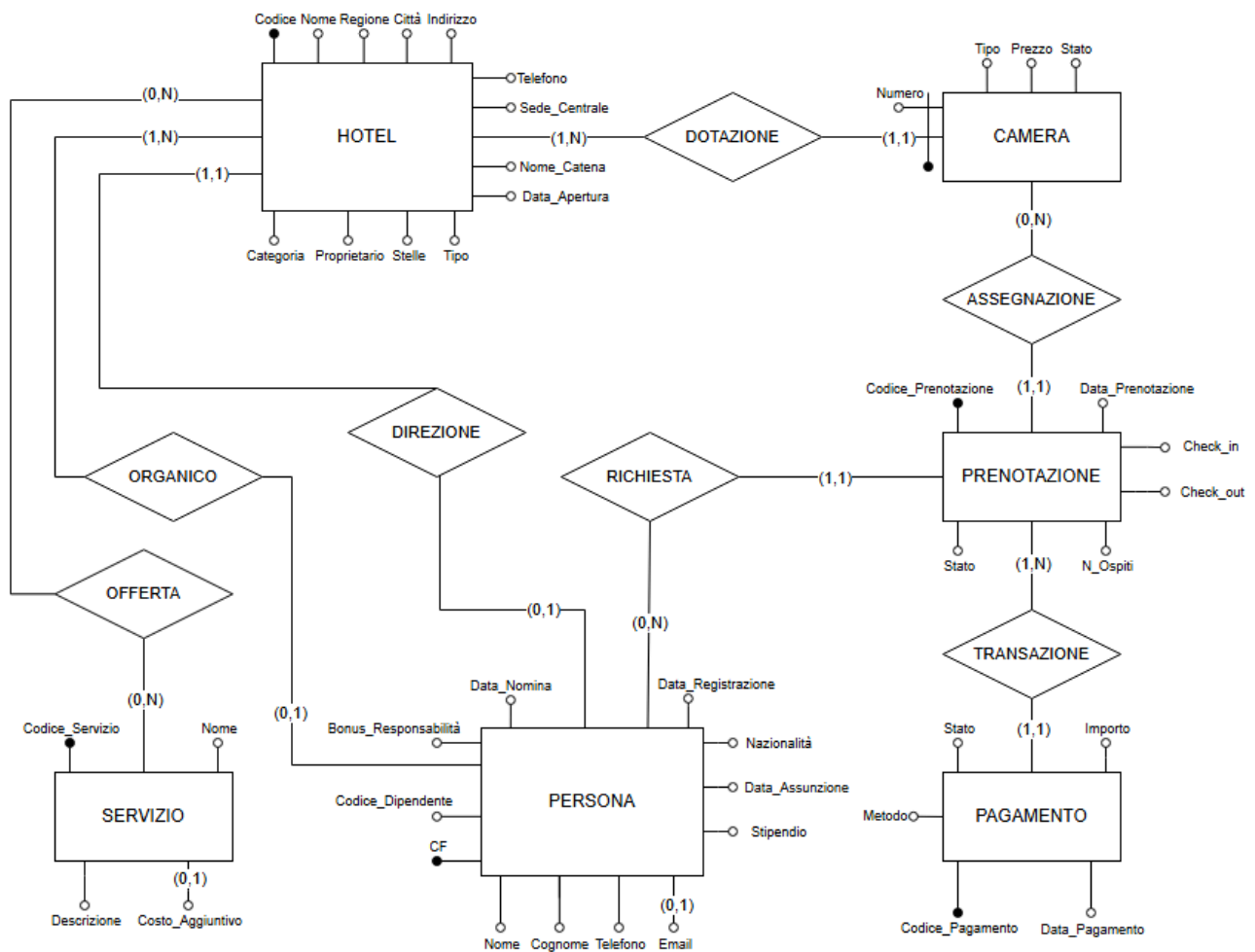


Figura 2: Diagramma E-R ristrutturato.

4.3 Schema Relazionale

Lo schema ristrutturato in Figura 2 contiene soltanto costrutti direttamente traducibili nel modello relazionale.

Di seguito viene riportato lo schema logico risultante. L'asterisco accanto al nome di un attributo indica che tale attributo può assumere valori nulli.

- **Hotel**(Codice, Nome, Indirizzo, Città, Regione, Stelle, Telefono, Tipo, Data_Apertura, Proprietario*, Categoria*, Nome_Catena*, Sede_Centrale*, Direttore)
- **Camera**(Hotel, Numero, Tipo, Prezzo, Stato)
- **Persona**(CF, Nome, Cognome, Telefono, Email*, Data_Registrazione*, Nazionalità*, Codice_Dipendente*, Data_Assunzione*, Stipendio*, Data_Nomina*, Bonus_Responsabilità*, Hotel*)
- **Prenotazione**(Codice_Prenotazione, Hotel, Camera, Data_Prenotazione, Check_In, Check_Out, N_Ospiti, Stato, Cliente)
- **Pagamento**(Codice_Pagamento, Data_Pagamento, Importo, Metodo, Stato, Prenotazione)

- **Servizio**(Codice_Servizio, Nome, Descrizione, Costo_Aggiuntivo*)
- **Offerta**(Hotel, Servizio)

Vincoli di integrità referenziale

- **Hotel.Direttore** → Persona.CF
- **Camera.Hotel** → Hotel.Codice
- **Persona.Hotel** → Hotel.Codice
- **Prenotazione.Cliente** → Persona.CF
- **Prenotazione.(Hotel, Camera)** → Camera.(Hotel, Numero)
- **Pagamento.Prenotazione** → Prenotazione.Codice_Prenotazione
- **Offerta.Hotel** → Hotel.Codice
- **Offerta.Servizio** → Servizio.Codice_Servizio

La tabella HOTEL contiene sia gli attributi generali della struttura sia gli attributi opzionali legati alla specifica tipologia di hotel. La distinzione tra le diverse casistiche viene gestita tramite l'attributo *Tipo* e tramite la nullabilità degli attributi non applicabili.

La tabella PERSONA raccoglie i dati anagrafici e gli attributi necessari a rappresentare i diversi ruoli presenti nel sistema. L'attributo *Hotel* indica la struttura presso cui lavora una persona nel caso in cui essa sia un dipendente o un direttore.

L'entità CAMERA viene tradotta in una tabella autonoma, mantenendo la dipendenza identificativa da HOTEL: il numero della camera è infatti univoco solo all'interno di uno stesso hotel, per cui la chiave primaria è composta da (*Hotel*, *Numero*).

La tabella OFFERTA deriva dall'associazione multi-a-multi tra HOTEL e SERVIZIO e contiene le chiavi esterne delle due tabelle coinvolte. La chiave primaria è composta dalla coppia (*Hotel*, *Servizio*), che identifica univocamente ciascun servizio offerto da un determinato hotel.

PRENOTAZIONE viene rappresentata come tabella autonoma e collega un cliente a una specifica camera in un determinato intervallo temporale. Per mantenere il riferimento corretto alla camera, vengono utilizzati gli attributi *Hotel* e *Camera*, che insieme fanno riferimento alla chiave primaria di CAMERA.

PAGAMENTO viene tradotta come tabella autonoma collegata a PRENOTAZIONE tramite una chiave esterna. In questo modo è possibile registrare uno o più pagamenti associati alla stessa prenotazione.

Una nota da considerare è che l'attributo *Direttore* in HOTEL è obbligatorio dal punto di vista logico, perché ogni hotel deve avere un direttore. In fase di implementazione, però, viene lasciato temporaneamente nullo durante il caricamento iniziale dei dati, per evitare un problema di dipendenza circolare tra Hotel e Persona: l'hotel fa riferimento al direttore, ma il direttore, essendo un dipendente, deve a sua volta essere associato a un hotel. Per questo motivo il direttore viene assegnato in un secondo momento, una volta inseriti tutti i dati.

5 Implementazione in PostgreSQL e Definizione delle Query

Il file *hotel.sql* contiene tutto il codice SQL necessario per la realizzazione della base di dati, inclusa la creazione delle tabelle, il popolamento con dati di esempio e la definizione delle query utilizzate per l'analisi dei dati.

Nel file sono inoltre presenti indici creati per ottimizzare le prestazioni di una delle interrogazioni.

5.1 Definizione delle Query

Query 1 Contare il numero di prenotazioni effettuate per ciascun hotel e visualizzarle in ordine decrescente rispetto al totale delle prenotazioni.

```
SELECT h.Nome, COUNT(p.Codice_Prenotazione) AS Totale_Prenotazioni
FROM Hotel h
JOIN Prenotazione p ON h.Codice = p.Hotel
GROUP BY h.Codice, h.Nome
ORDER BY Totale_Prenotazioni DESC;
```

	nome character varying (100)	totale_prenotazioni bigint
1	Grand Milano	4
2	Etna Resort	3
3	Hotel Vesuvio	2
4	Royal Firenze	2
5	Laguna Blu	2

Query 2 Calcolare l'incasso totale generato da ciascun hotel considerando solo i pagamenti effettuati e ordinare i risultati dal valore più alto al più basso.

```
SELECT h.Nome, SUM(pa.Importo) AS Incasso_Totale
FROM Hotel h
JOIN Prenotazione pr ON h.Codice = pr.Hotel
JOIN Pagamento pa ON pr.Codice_Prenotazione = pa.Prenotazione
WHERE pa.Stato = 'Effettuato'
AND pr.Stato IN ('Confermata', 'In_Corso', 'Conclusa')
GROUP BY h.Codice, h.Nome
ORDER BY Incasso_Totale DESC;
```

	nome character varying (100)	incasso_totale numeric
1	Etna Resort	7160.00
2	Grand Milano	6710.00
3	Laguna Blu	5080.00
4	Torino Palace	2800.00
5	Royal Firenze	1820.00

Query 3 Determinare il numero medio di ospiti per prenotazione in ciascun hotel e mostrare i risultati in ordine decrescente.

```
SELECT h.Nome, AVG(pr.N_Ospiti) AS Media_Ospiti
FROM Hotel h
JOIN Prenotazione pr ON h.Codice = pr.Hotel
GROUP BY h.Codice, h.Nome
ORDER BY Media_Ospiti DESC;
```

	nome character varying (100)	media_ospiti numeric
1	Hotel Adriatico	4.0000000000000000
2	Torino Palace	2.5000000000000000
3	Laguna Blu	2.5000000000000000
4	Royal Firenze	2.0000000000000000
5	Dolomiti Resort	2.0000000000000000

Query 4 Trovare gli hotel che offrono più di 3 servizi e visualizzare il numero totale di servizi disponibili per ciascuno.

```
SELECT h.Nome, COUNT(o.Servizio) AS Numero_Servizi
FROM Hotel h
JOIN Offerta o ON h.Codice = o.Hotel
GROUP BY h.Codice, h.Nome
HAVING COUNT(o.Servizio) > 3
ORDER BY Numero_Servizi DESC;
```

	nome character varying (100)	numero_servizi bigint
1	Etna Resort	5
2	Torino Palace	5
3	Grand Milano	4
4	Dolomiti Resort	4
5	Royal Firenze	4

Query 5 Contare il numero di dipendenti che lavorano in ciascun hotel e ordinare i risultati in ordine decrescente rispetto al totale dei dipendenti.

```
SELECT h.Nome, COUNT(p.CF) AS Numero_Dipendenti
FROM Hotel h
JOIN Persona p ON h.Codice = p.Hotel
WHERE p.Codice_Dipendente IS NOT NULL
GROUP BY h.Codice, h.Nome
ORDER BY Numero_Dipendenti DESC;
```

	nome character varying (100) 🔒	numero_dipendenti bigint 🔒
1	Grand Milano	6
2	Laguna Blu	5
3	Hotel Vesuvio	4
4	Dolomiti Resort	4
5	Etna Resort	4

Query 6 Trovare le stanze di hotel libere che hanno prezzo per notte minore o uguale ad un determinato valore (ad esempio 150).

```
SELECT h.Nome AS Hotel, c.Numero, c.Tipo, c.Prezzo
FROM Hotel h
JOIN Camera c ON h.Codice = c.Hotel
WHERE c.Stato = 'Libera'
      AND c.Prezzo <= 150
ORDER BY c.Prezzo ASC;
```

	hotel character varying (100) 🔒	numero integer 🔒	tipo character varying (30) 🔒	prezzo numeric (8,2) 🔒
1	Hotel Adriatico	101	Singola	80.00
2	Hotel Lago	101	Singola	90.00
3	Hotel Vesuvio	101	Singola	95.00
4	Etna Resort	101	Singola	100.00
5	Grand Milano	101	Singola	120.00
6	Royal Firenze	101	Singola	130.00

5.2 Creazione degli indici

Indici significativi possono essere creati sulla query 2, perché è una tra le più costose, poiché coinvolge tre tabelle e filtra i pagamenti in base allo stato.

```
CREATE INDEX idx_pagamento_stato_prenotazione ON Pagamento (Stato, Prenotazione);
```

```
CREATE INDEX idx_prenotazione_hotel ON Prenotazione (Hotel);
```

Il primo indice, sulla tabella PAGAMENTO (*Stato*, *Prenotazione*), permette di individuare più rapidamente i pagamenti effettuati e velocizza le operazioni di JOIN con la tabella PRENOTAZIONE. Il secondo indice, sull'attributo *Hotel* della tabella PRENOTAZIONE, migliora le operazioni di JOIN tra PRENOTAZIONE e HOTEL, riducendo il numero di accessi necessari durante l'esecuzione della query. La scelta del tipo B-tree è dovuta alla sua elevata efficienza nelle query con filtri, JOIN e aggregazioni.

Il miglioramento sui tempi di esecuzione può essere notato su database con grandi quantità di dati, ma con database di piccola dimensione questo non può essere garantito.

6 Applicazione Software

Il file *query.c* contiene il codice C che permette di collegarsi al database PostgreSQL. Dopo aver stabilito la connessione, il programma mostra un menu da cui l'utente può scegliere una delle query definite precedentemente. In base alla scelta effettuata, viene eseguita la query corrispondente e il risultato viene stampato a video.

La query 6 è stata resa parametrica: quando viene selezionata, l'utente deve inserire un importo massimo. Il programma mostrerà quindi le camere libere con prezzo minore o uguale a tale valore.

Prima di compilare ed eseguire *query.c*, è necessario modificare i parametri di connessione al database PostgreSQL presenti all'inizio del file sorgente con quelli relativi alla propria installazione di PostgreSQL:

```
6  #define PG_HOST "127.0.0.1"
7  #define PG_USER "postgres"
8  #define PG_DB "hotel_db"
9  #define PG_PASS "****"
10 #define PG_PORT 5432
```